

使用 PostgreSQL 適用的 Cloud SQL，將完整堆疊 JavaScript 應用程式部署至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/deploy-application-with-database/cloud-sql-nextjs?hl=zh-tw>

步驟數：12

目錄

1. [總覽](#)
2. [必要條件](#)
3. [專案設定](#)
4. [開啟 Cloud Shell 編輯器](#)
5. [啟用 API](#)
6. [設定服務帳戶](#)
7. [建立 Cloud SQL 資料庫](#)
8. [準備應用程式](#)
9. [將應用程式部署至 Cloud Run](#)
10. [使用 Gemini Code Assist 新增功能](#)
11. [將應用程式重新部署至 Cloud Run](#)
12. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 總覽

[Cloud Run](#) 是一個全代管平台，讓您能直接在可擴充的 Google 基礎架構上執行程式碼。本程式碼研究室將說明如何將 Cloud Run 上的 Next.js 應用程式連線至 PostgreSQL 適用的 [Cloud SQL](#) 資料庫。

本實驗室的學習內容包括：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
 - 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫
-

步驟 2 · 約 0 分鐘

2. 必要條件

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
-

步驟 3 · 約 0 分鐘

3. 專案設定

1. 登入 [Google Cloud 控制台](#)。
 2. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 3. [建立新專案](#)，或選擇重複使用現有專案。
-

步驟 4 · 約 0 分鐘

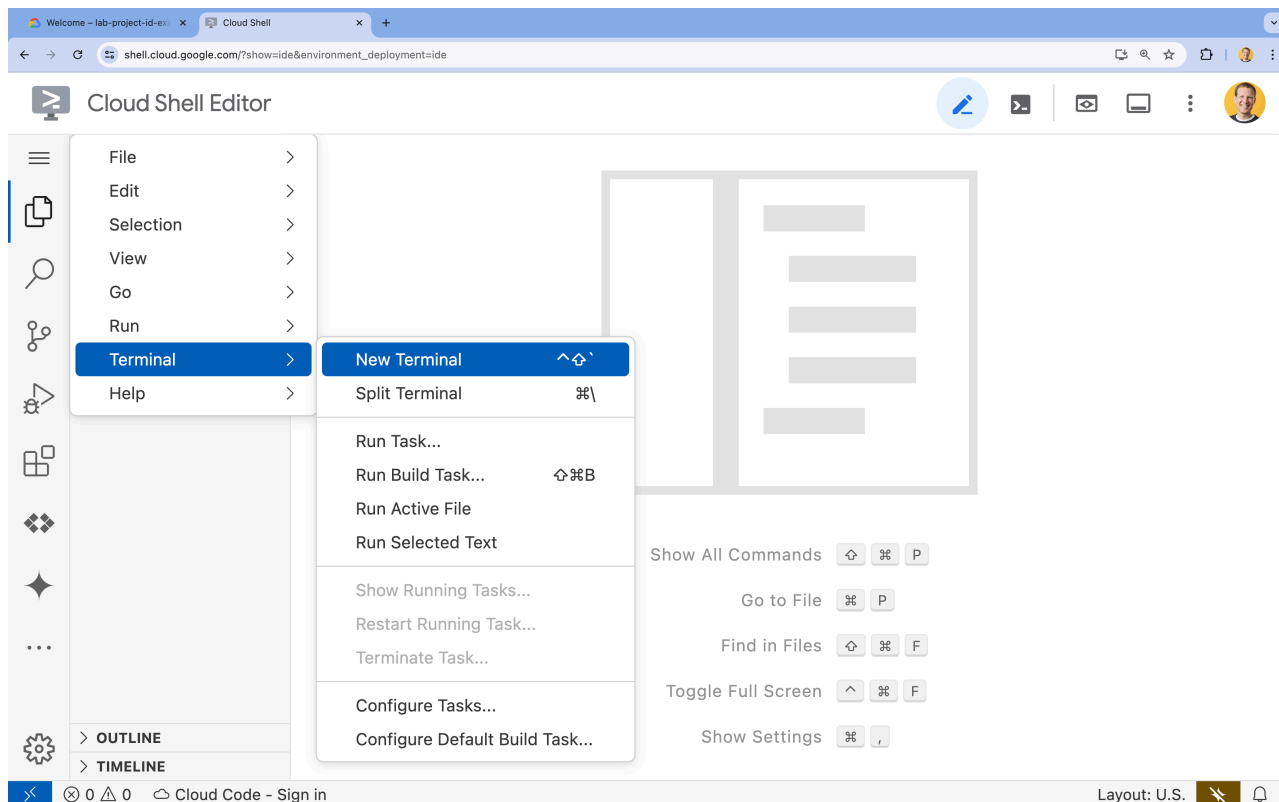
4. 開啟 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

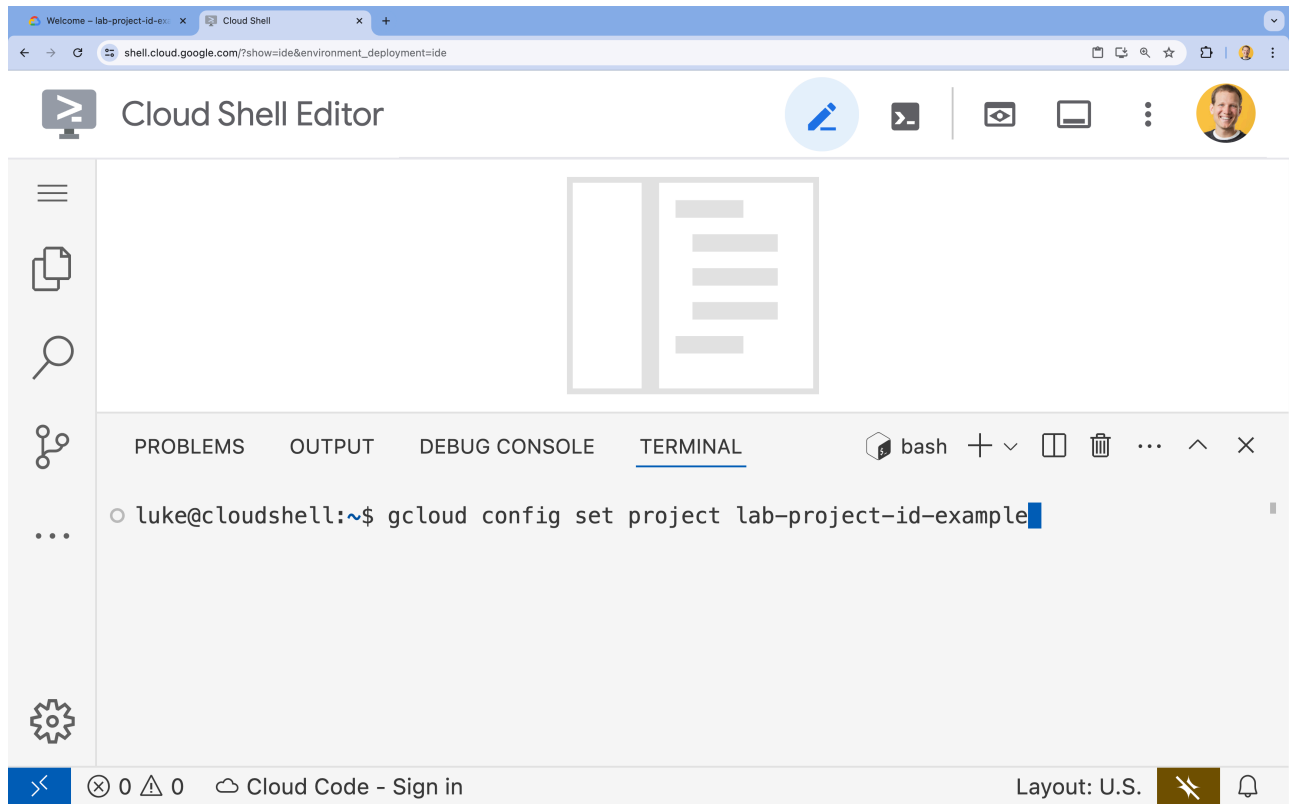
- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 5 · 約 0 分鐘

5. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  compute.googleapis.com \  
  sqladmin.googleapis.com \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  networkconnectivity.googleapis.com \  
  servicenetworking.googleapis.com \  
  cloudaicompanion.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

6. 設定服務帳戶

建立及設定 Google Cloud 服務帳戶，供 Cloud Run 使用，確保該帳戶具備連線至 Cloud SQL 的正確權限。

1. 執行下列 `gcloud iam service-accounts create` 指令，建立新的服務帳戶：

```
gcloud iam service-accounts create quickstart-service-account \
  --display-name="Quickstart Service Account"
```

2. 執行下列 `gcloud projects add-iam-policy-binding` 指令，將「記錄檔寫入者」角色新增至您剛建立的 Google Cloud 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member="serviceAccount:quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role="roles/logging.logWriter"
```

7. 建立 Cloud SQL 資料庫

1. 建立服務連線政策，允許 Cloud Run 透過 [Private Service Connect](#) 連線至 Cloud SQL

```
gcloud network-connectivity service-connection-policies create quickstart-policy \
  --network=default \
  --project=${GOOGLE_CLOUD_PROJECT} \
  --region=us-central1 \
  --service-class=google-cloud-sql \
  --
subnets=https://www.googleapis.com/compute/v1/projects/${GOOGLE_CLOUD_PROJECT}/regions/us
-central1/subnetworks/default
```

2. 為資料庫產生專屬密碼

```
export DB_PASSWORD=$(openssl rand -base64 20)
```

3. 執行 `gcloud sql instances create` 指令，建立 Cloud SQL 執行個體

```
gcloud sql instances create quickstart-instance \
  --project=${GOOGLE_CLOUD_PROJECT} \
  --root-password=${DB_PASSWORD} \
  --database-version=POSTGRES_17 \
  --tier=db-perf-optimized-N-2 \
  --region=us-central1 \
  --ssl-mode=ENCRYPTED_ONLY \
  --no-assign-ip \
  --enable-private-service-connect \
  --psc-auto-
connections=network=projects/${GOOGLE_CLOUD_PROJECT}/global/networks/default
```

這個指令可能需要幾分鐘才能完成。

1. 執行 `gcloud sql databases create` 指令，在 `quickstart-instance` 中建立 Cloud SQL 資料庫。

```
gcloud sql databases create quickstart_db \
  --instance=quickstart-instance
```

步驟 8 · 約 0 分鐘

8. 準備應用程式

準備回應 HTTP 要求的 Next.js 應用程式。

1. 如要建立名為 `task-app` 的新 Next.js 專案，請使用下列指令：

```
npx --yes create-next-app@15 task-app \
  --ts \
  --eslint \
  --tailwind \
  --no-src-dir \
  --turbo \
  --app \
  --no-import-alias
```

2. 將目錄變更為 `task-app`：

```
cd task-app
```

3. 安裝 `pg`，與 PostgreSQL 資料庫互動。

```
npm install pg
```

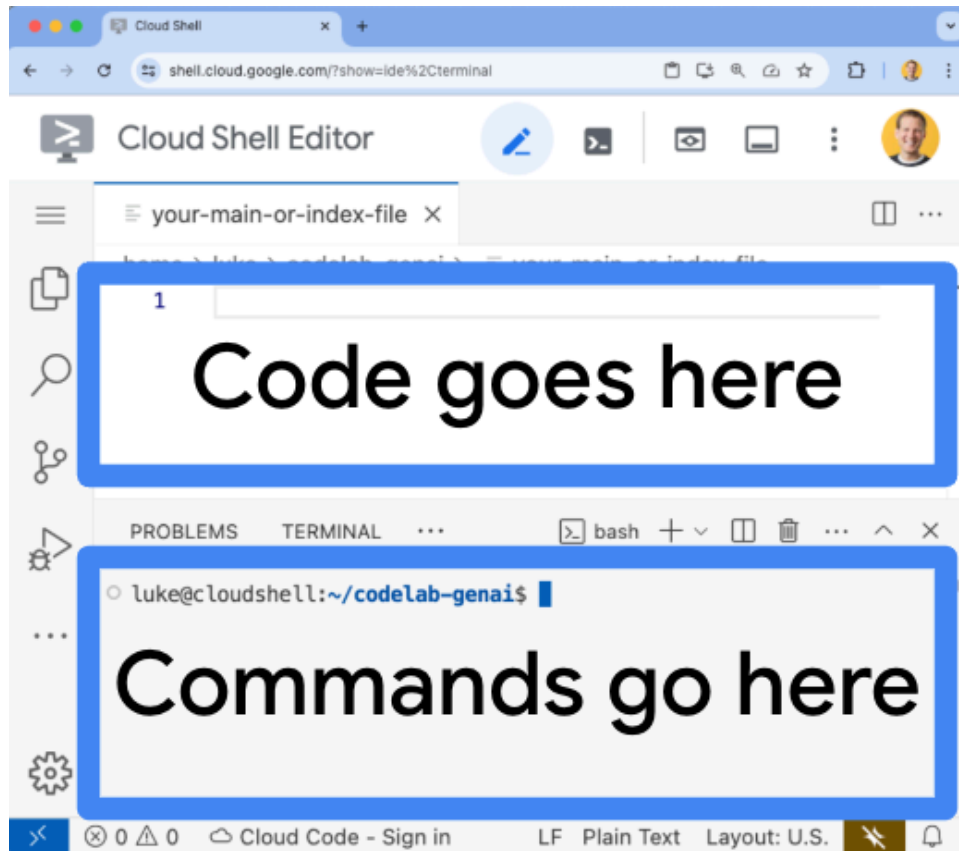
4. 安裝 `@types/pg` 做為開發依附元件，以使用 TypeScript Next.js 應用程式。

```
npm install --save-dev @types/pg
```

5. 在 Cloud Shell 編輯器中開啟 `actions.ts` 檔案：

```
cloudshell edit app/actions.ts
```

畫面頂端現在應該會顯示空白檔案。您可以在這裡編輯這個 `actions.ts` 檔案。



6. 複製下列程式碼，並貼到開啟的 `actions.ts` 檔案中：

```

'use server'
import pg from 'pg';

type Task = {
  id: string;
  title: string;
  status: 'IN_PROGRESS' | 'COMPLETE';
};

const { Pool } = pg;

const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  ssl: {
    // @ts-expect-error require true is not recognized by @types/pg, but does exist on pg
    require: true,
    rejectUnauthorized: false, // required for self-signed certs
    // https://node-postgres.com/features/ssl#self-signed-cert
  }
});

const tableCreationIfDoesNotExist = async () => {
  await pool.query(`CREATE TABLE IF NOT EXISTS tasks (
    id SERIAL NOT NULL,
    created_at timestamp NOT NULL,
    status VARCHAR(255) NOT NULL default 'IN_PROGRESS',
    title VARCHAR(1024) NOT NULL,
    PRIMARY KEY (id)
  );`);
}

// CREATE
export async function addNewTaskToDatabase(newTask: string) {
  await tableCreationIfDoesNotExist();
  await pool.query(`INSERT INTO tasks(created_at, status, title) VALUES(NOW(),
'IN_PROGRESS', $1)`, [newTask]);
  return;
}

// READ
export async function getTasksFromDatabase() {
  await tableCreationIfDoesNotExist();
  const { rows } = await pool.query(`SELECT id, created_at, status, title FROM tasks
ORDER BY created_at DESC LIMIT 100`);
  return rows;
}

// UPDATE
export async function updateTaskInDatabase(task: Task) {

```

```

await tableCreationIfDoesNotExist();
await pool.query(
  `UPDATE tasks SET status = $1, title = $2 WHERE id = $3`,
  [task.status, task.title, task.id]
);
return;
}

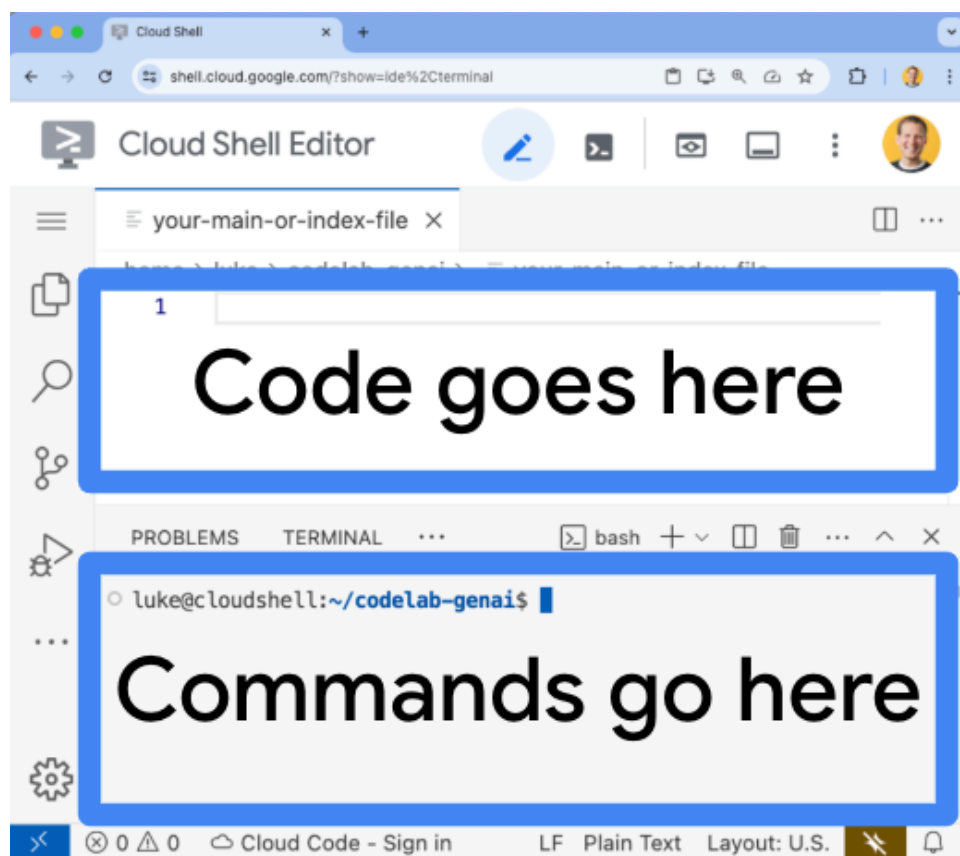
// DELETE
export async function deleteTaskFromDatabase(taskId: string) {
  await tableCreationIfDoesNotExist();
  await pool.query(`DELETE FROM tasks WHERE id = $1`, [taskId]);
  return;
}

```

7. 在 Cloud Shell 編輯器中開啟 `page.tsx` 檔案：

```
cloudshell edit app/page.tsx
```

畫面頂端現在應該會顯示現有檔案。您可以在這裡編輯這個 `page.tsx` 檔案。



8. 刪除 `page.tsx` 檔案的現有內容。

9. 複製下列程式碼，並貼到開啟的 `page.tsx` 檔案中：

```

'use client'
import React, { useEffect, useState } from "react";
import { addNewTaskToDatabase, getTasksFromDatabase, deleteTaskFromDatabase,
updateTaskInDatabase } from "../actions";

type Task = {
  id: string;
  title: string;
  status: 'IN_PROGRESS' | 'COMPLETE';
};

export default function Home() {
  const [newTaskTitle, setNewTaskTitle] = useState('');
  const [tasks, setTasks] = useState<Task[]>([]);

  async function getTasks() {
    const updatedListOfTasks = await getTasksFromDatabase();
    setTasks(updatedListOfTasks);
  }

  useEffect(() => {
    getTasks();
  }, []);

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    await addNewTaskToDatabase(newTaskTitle);
    await getTasks();
    setNewTaskTitle('');
  };

  async function updateTask(task: Task, newTaskValues: Partial<Task>) {
    await updateTaskInDatabase({ ...task, ...newTaskValues });
    await getTasks();
  }

  async function deleteTask(taskId: string) {
    await deleteTaskFromDatabase(taskId);
    await getTasks();
  }

  return (
    <main className="p-4">
      <h2 className="text-2xl font-bold mb-4">To Do List</h2>
      <div className="flex mb-4">
        <form onSubmit={handleSubmit} className="flex mb-8">
          <input
            type="text"
            placeholder="New Task Title"
            value={newTaskTitle}
            onChange={(e) => setNewTaskTitle(e.target.value)}
            className="flex-grow border border-gray-400 rounded px-3 py-2 mr-2 bg-

```

```

inherit"
    />
    <button
      type="submit"
      className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4
rounded text-nowrap"
    >
      Add New Task
    </button>
  </form>
</div>
<table className="w-full">
  <tbody>
    {tasks.map(function (task) {
      const isComplete = task.status === 'COMPLETE';
      return (
        <tr key={task.id} className="border-b border-gray-200">
          <td className="py-2 px-4">
            <input
              type="checkbox"
              checked={isComplete}
              onClick={() => updateTask(task, { status: isComplete ? 'IN_PROGRESS'
: 'COMPLETE' })}
              className="transition-transform duration-300 ease-in-out transform
scale-100 checked:scale-125 checked:bg-green-500"
            />
          </td>
          <td className="py-2 px-4">
            <span
              className={`transition-all duration-300 ease-in-out ${isComplete ?
'line-through text-gray-400 opacity-50' : 'opacity-100'}`}
            >
              {task.title}
            </span>
          </td>
          <td className="py-2 px-4">
            <button
              onClick={() => deleteTask(task.id)}
              className="bg-red-500 hover:bg-red-700 text-white font-bold py-2 px-4
rounded float-right"
            >
              Delete
            </button>
          </td>
        </tr>
      );
    })}
  </tbody>
</table>
</main>
);
}

```

應用程式現在已可部署。

9. 將應用程式部署至 Cloud Run

1. 執行 `gcloud projects add-iam-policy-binding` 指令，將「網路使用者」角色新增至即將建立的 Cloud Run 服務的 Cloud Run 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member "serviceAccount:service-${(gcloud projects describe ${GOOGLE_CLOUD_PROJECT} -
  --format="value(projectNumber)")@serverless-robot-prod.iam.gserviceaccount.com" \
  --role "roles/compute.networkUser"
```

1. 執行 `gcloud projects add-iam-policy-binding` 指令，為即將建立的 Cloud Run 服務，將「Artifact Registry 寫入者」角色新增至目前使用者。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member=user:${(gcloud auth list --filter=status:ACTIVE --format="value(account)")} \
  --role="roles/artifactregistry.writer"
```

1. 執行下列指令，將應用程式部署至 Cloud Run：

```
gcloud run deploy helloworld \
  --region=us-central1 \
  --source=. \
  --set-env-vars DB_NAME="quickstart_db" \
  --set-env-vars DB_USER="postgres" \
  --set-env-vars DB_PASSWORD=${DB_PASSWORD} \
  --set-env-vars DB_HOST="$(gcloud sql instances describe quickstart-instance --
  project=${GOOGLE_CLOUD_PROJECT} --
  format='value(settings.ipConfiguration.pscConfig.pscAutoConnections.ipAddress)')" \
  --service-account="quickstart-service-
  account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --network=default \
  --subnet=default \
  --allow-unauthenticated
```

2. 如果系統提示您確認是否要繼續，請按下 `y` 和 `Enter`：

```
Do you want to continue (Y/n)? Y
```

幾分鐘後，應用程式應會提供網址供您造訪。

前往該網址，即可查看應用程式的運作情形。每次造訪網址或重新整理頁面時，都會看到工作應用程式。

10. 使用 Gemini Code Assist 新增功能

您現在已部署含有資料庫的網頁應用程式。接著，我們使用 AI 輔助功能，在 next.js 應用程式中新增功能。

1. 返回 Cloud Shell 編輯器
2. 再次開啟「`page.tsx`」

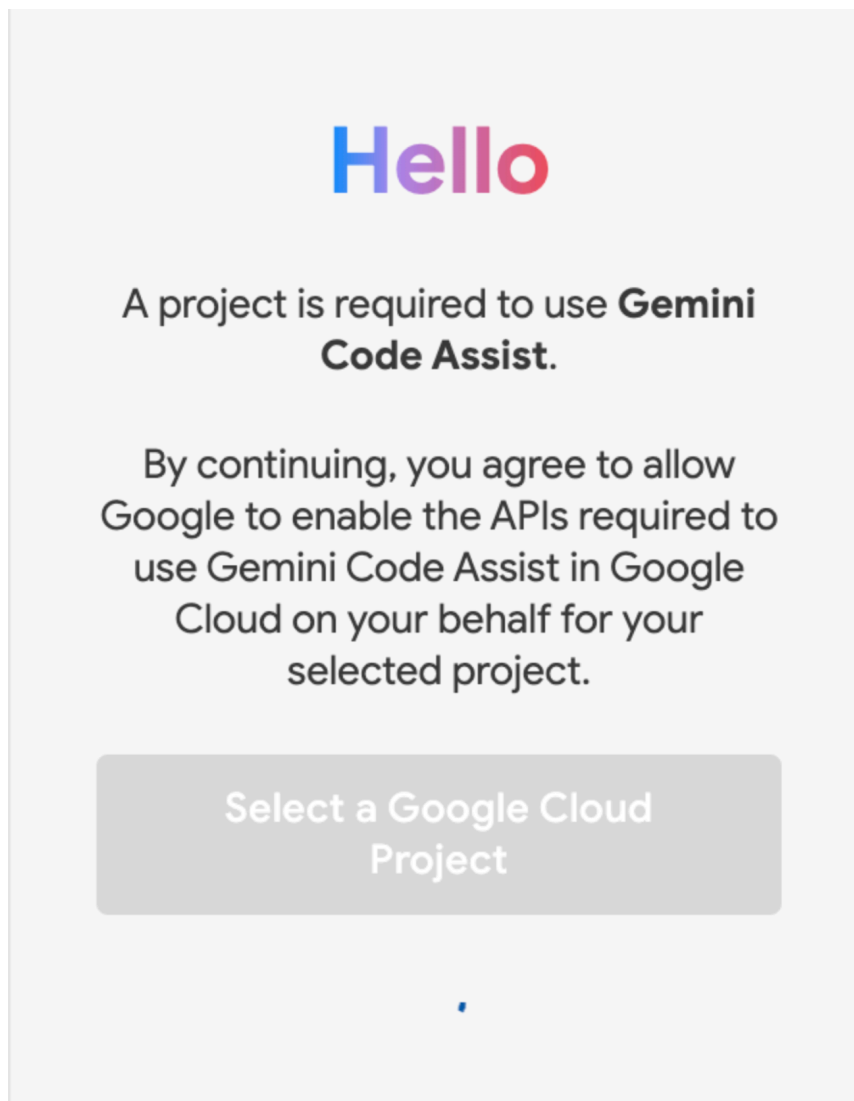
```
cd ~/task-app  
cloudshell edit app/page.tsx
```

3. 在 Cloud Shell 編輯器中前往 Gemini Code Assist：

- 按一下畫面左側工具列中的 Gemini 圖示



- 如果出現提示，請輸入 Google 帳戶憑證登入
- 如果系統提示選取專案，請選取您為這個程式碼研究室建立的專案



4. 輸入提示：`Add the ability to update the title of the task. The code in your output should be complete and working code.`。回覆內容應包含類似下列程式碼片段，以新增 `handleEditStart` 和

handleEditCancel 函式：

```
const [editingTaskId, setEditingTaskId] = useState('');
const [editedTaskTitle, setEditedTaskTitle] = useState('');

function handleEditStart(task: Task) {
  setEditingTaskId(task.id);
  setEditedTaskTitle(task.title);
};
```

5. 將 `page.tsx` 換成 Gemini Code Assist 的輸出內容。以下是可用的範例：

```

'use client'
import React, { useEffect, useState } from "react";
import { addNewTaskToDatabase, getTasksFromDatabase, deleteTaskFromDatabase,
updateTaskInDatabase } from "../actions";

type Task = {
  id: string;
  title: string;
  status: 'IN_PROGRESS' | 'COMPLETE';
};

export default function Home() {
  const [newTaskTitle, setNewTaskTitle] = useState('');
  const [tasks, setTasks] = useState<Task[]>([]);
  const [editingTaskId, setEditingTaskId] = useState('');
  const [editedTaskTitle, setEditedTaskTitle] = useState('');

  async function getTasks() {
    const updatedListOfTasks = await getTasksFromDatabase();
    setTasks(updatedListOfTasks);
  }

  useEffect(() => {
    getTasks();
  }, []);

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    await addNewTaskToDatabase(newTaskTitle);
    await getTasks();
    setNewTaskTitle('');
  };

  async function updateTask(task: Task, newTaskValues: Partial<Task>) {
    await updateTaskInDatabase({ ...task, ...newTaskValues });
    await getTasks();
    setEditingTaskId('');
    setEditedTaskTitle('');
  }

  async function deleteTask(taskId: string) {
    await deleteTaskFromDatabase(taskId);
    await getTasks();
  }

  function handleEditStart(task: Task) {
    setEditingTaskId(task.id);
    setEditedTaskTitle(task.title);
  };

  return (
    <main className="p-4">

```



```

2 bg-inherit"
    />
  </form>
) : (
  <span
    onClick={() => handleEditStart(task)}
    className={`transition-all duration-300 ease-in-out cursor-pointer
${isComplete ? 'line-through text-gray-400 opacity-50' : 'opacity-100'}`}
  >
    {task.title}
  </span>
)}
</td>
<td className="py-2 px-4">
  <button
    onClick={() => deleteTask(task.id)}
    className="bg-red-500 hover:bg-red-700 text-white font-bold py-2 px-4
rounded float-right"
  >
    Delete
  </button>
</td>
</tr>
);
}}
</tbody>
</table>
</main>
);
}

```

11. 將應用程式重新部署至 Cloud Run

1. 執行下列指令，將應用程式部署至 Cloud Run：

```
gcloud run deploy helloworld \
  --region=us-central1 \
  --source=. \
  --set-env-vars DB_NAME="quickstart_db" \
  --set-env-vars DB_USER="postgres" \
  --set-env-vars DB_PASSWORD=${DB_PASSWORD} \
  --set-env-vars DB_HOST="$(gcloud sql instances describe quickstart-instance --
project=${GOOGLE_CLOUD_PROJECT} --
format='value(settings.ipConfiguration.pscConfig.pscAutoConnections.ipAddress)')" \
  --service-account="quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --network=default \
  --subnet=default \
  --allow-unauthenticated
```

2. 如果系統提示您確認是否要繼續，請按下 **Y** 和 **Enter**：

```
Do you want to continue (Y/n)? Y
```

12. 恭喜

在本實驗室中，您已學會如何執行下列工作：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
- 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫

清除所用資源

Cloud SQL 沒有免費方案，如果您繼續使用，系統會向您收費。您可以刪除 Cloud 專案，以免產生額外費用。

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。刪除 Cloud 專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/task-app
```

2. 警告！這項操作無法復原，如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存到其他位置。

```
sudo rm -rf $HOME
```

持續掌握情況

- 使用 [Cloud SQL Node.js 連接器](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
 - 使用 [Cloud SQL Node.js 連接器](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
 - 使用 [Node.js Admin SDK](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配使用 Firestore
 - 使用 [Node.js Admin SDK](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 Firestore 使用
-